

GUÍA COMPLETA — Despliegue de Frontend Angular en la Nube con Docker

1. Preparar el proyecto Angular

Ubícate en la carpeta raíz del proyecto Angular (donde está el archivo `angular.json`).

Ejecuta el siguiente comando para asegurarte de tener las dependencias actualizadas:

```
npm install
```

Compila el proyecto en modo producción:

```
ng build --configuration production
```

Esto generará una carpeta llamada `dist/` dentro del proyecto, donde estará la versión optimizada del frontend lista para desplegar.

2. Crear el archivo Dockerfile para Angular

Dentro de la raíz del proyecto (donde está `package.json`), crea un archivo llamado Dockerfile con el siguiente contenido:

```
# Etapa 1: Compilar Angular
FROM node:18 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build --configuration production

# Etapa 2: Servir con NGINX
FROM nginx:alpine
COPY --from=build /app/dist/<nombre-de-tu-proyecto> /usr/share/nginx/html

# Configuración de NGINX
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- ♦ Reemplaza `<nombre-de-tu-proyecto>` por el nombre real que aparece en tu carpeta `dist/`.
-

3. Crear el archivo docker-compose.yml

Este archivo orquesta los servicios: frontend, backend y base de datos.

Ejemplo completo:

```
version: "3.8"
```

```
services:
```

```

frontend:
  build: ./frontend
  container_name: angular-frontend
  ports:
    - "4200:80"
  depends_on:
    - backend

backend:
  build: ./backend
  container_name: spring-backend
  ports:
    - "8080:8080"
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/mi_base_datos
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: 12345
  depends_on:
    - db

db:
  image: mysql:8.0
  container_name: mysql-db
  restart: always
  environment:
    MYSQL_ROOT_PASSWORD: 12345
    MYSQL_DATABASE: mi_base_datos
  ports:
    - "3306:3306"

```

Estructura recomendada del proyecto

```

/mi-proyecto
├── /frontend
│   ├── Dockerfile
│   ├── angular.json
│   └── src/
├── /backend
│   ├── Dockerfile
│   ├── src/
│   └── pom.xml
└── docker-compose.yml

```

4. Desplegar en la nube (opciones comunes)

Opción 1: Render.com

1. Crea una cuenta en <https://render.com>.
2. Conecta tu repositorio (GitHub/GitLab).
3. Sube tu proyecto con el `Dockerfile` y `docker-compose.yml`.

4. Render detectará automáticamente los servicios y los desplegará.
5. Asigna variables de entorno (por ejemplo, contraseñas de base de datos).

Opción 2: AWS EC2 (manual con Docker)

1. Crea una instancia EC2 (Ubuntu).
2. Instala Docker y Docker Compose:

```
sudo apt update
sudo apt install docker.io docker-compose -y
```

3. Clona tu proyecto desde GitHub:

```
git clone https://github.com/usuario/mi-proyecto.git
cd mi-proyecto
```

4. Ejecuta:

```
docker-compose up -d
```

5. Accede a tu IP pública en el navegador:

```
http://<tu-ip-ec2>:4200
```

Opción 3: Google Cloud Run / Azure App Service

Ambas plataformas permiten desplegar imágenes Docker directamente.
Solo debes construir la imagen y subirla:

```
docker build -t mi-frontend .
docker push gcr.io/<mi-proyecto>/mi-frontend
```

Luego, crea un servicio en Cloud Run y conéctalo a esa imagen.

5. Verificar que todo esté corriendo

Verifica los contenedores activos:

```
docker ps
```

Deberías ver algo como:

CONTAINER ID	IMAGE	PORTS	NAMES
a1b2c3d4e5f6	angular-frontend	0.0.0.0:4200->80/tcp	angular-frontend
b2c3d4e5f6a1	spring-backend	0.0.0.0:8080->8080/tcp	spring-backend
c3d4e5f6a1b2	mysql:8.0	0.0.0.0:3306->3306/tcp	mysql-db

Accede desde el navegador:

- Frontend: <http://localhost:4200>
- Backend: <http://localhost:8080>

- Base de datos: conectada internamente por Docker
-

6. Detener los servicios

Cuando termines, puedes apagar todos los contenedores con:

```
docker-compose down
```

Si deseas eliminar también las imágenes y volúmenes:

```
docker-compose down --rm all -v
```

7. Consejos finales

- Usa variables de entorno (`.env`) para contraseñas y configuraciones sensibles.
- En producción, configura HTTPS (puedes hacerlo con un proxy NGINX o certificado de Let's Encrypt).
- Monitorea tus contenedores con:

```
docker logs -f angular-frontend  
docker logs -f spring-backend
```

- Si usas dominios personalizados, configura un DNS apuntando a la IP pública de tu servidor.
-

Resultado final

Tu entorno en la nube tendrá:

- Frontend Angular servido con NGINX.
- Backend Spring Boot corriendo en un contenedor separado.
- Base de datos MySQL/PostgreSQL persistente.
- Todo orquestado con Docker Compose y accesible desde Internet.